Selective Disclosure — ACL / Add Viewer

> ***Nox Protocol — Developer Guide*** *Arbitrum Sepolia Testnet (chainId: 421614)*

Overview

Selective Disclosure allows a cToken holder to grant a third party (auditor, regulator, business partner) read access to their encrypted balance — without revealing it publicly on-chain.

This is done via the `addViewer` function on the **NoxCompute** proxy contract. The viewer can then call `decrypt()` via the HandleClient to read the balance value.

Use case: A company holds 10,000 cRLC and needs to prove their reserves to an auditor. They grant the auditor's address viewer access. The auditor can now decrypt and verify the balance — no one else can.

Architecture

Component	Path / Address
Hook	`hooks/use-add-viewer.ts`
Modal	`components/modals/selective-disclosure-modal.tsx`
NoxCompute ABI	`lib/nox-compute-abi.ts`
cToken ABI	`lib/confidential-token-abi.ts` (for `confidentialBalanceOf`)
Gas Utility	`lib/gas.ts`
NoxCompute Contract	`0x5633472D35E18464CA24Ab974954fB3b1B122eA6`
cRLC Contract	`0x271f46e78f2fe59817854dabde47729ac4935765`

Step-by-Step Flow

```
User                cToken (cRLC)          NoxCompute
  |                     |                      |
  |-- 1. confidentialBalanceOf(user) ---------->|
  |<--- handle (bytes32) ---|                  |
  |                     |                      |
  |   [repeat for each cToken]                 |
  |                     |                      |
  |-- 2. addViewer(handle, viewer) ------------>|
  |<--- tx receipt ----------------------------|
  |                     |                      |
  |   [repeat for each non-zero handle]        |
  |                     |                      |
  | Viewer can now decrypt the balance         |
```

Step 0 — Validation

```
import { isAddress } from "viem";

// Wallet must be connected
if (!address) throw new Error("Wallet not connected");

// Viewer address must be valid
if (!isAddress(viewerAddress)) throw new Error("Invalid viewer address");

// Public client must be available
if (!publicClient) throw new Error("Public client not available");
```

Step 1 — Read Balance Handles

For each confidential token the user might hold, read the current balance handle from the cToken contract.

```
// Called on the cToken contract (e.g. cRLC)
function confidentialBalanceOf(address account) external view returns (bytes32);
```

Implementation:

```
import { ZERO_HANDLE } from "@/lib/contracts";

const handles: { token: TokenConfig; handle: `0x${string}` }[] = [];

for (const token of tokens) {
  const handle = await publicClient.readContract({
    address: token.confidentialAddress,
    abi: confidentialTokenAbi,
    functionName: "confidentialBalanceOf",
    args: [address],     // User's wallet address
  });

  // Skip tokens with no balance (zero handle = never wrapped)
  if (handle !== ZERO_HANDLE) {
    handles.push({ token, handle });
  }
}
```

Zero Handle Check:

```
const ZERO_HANDLE = "0x" + "0".repeat(64);
// 0x0000000000000000000000000000000000000000000000000000000000000000
```

If all handles are zero, the user has no confidential balances → show error: *"Wrap tokens first before granting viewer access."*

Step 2 — Grant Viewer Access

For each non-zero handle, call `addViewer` on the NoxCompute contract.

```
// Called on the NoxCompute proxy contract
function addViewer(bytes32 handle, address viewer) external;
```

Parameters:

Param	Value	Description
`handle`	Balance handle from Step 1	The encrypted balance reference
`viewer`	Auditor/viewer address	The address being granted read access

Implementation:

```
for (const { handle } of handles) {
  const overrides = await estimateGasOverrides(publicClient);

  const txHash = await writeContractAsync({
    address: NOX_COMPUTE_ADDRESS,
    abi: noxComputeAbi,
    functionName: "addViewer",
    args: [handle, viewerAddress],
    ...overrides,
  });

  await waitForTransactionReceipt(config, { hash: txHash });

  // 2-second cooldown between sequential calls
  await new Promise(r => setTimeout(r, 2000));
}
```

State Machine

```
idle → reading-handle → granting → confirmed
              ↘              ↘
             error          error
```

State	UI	Description
`idle`	Form visible	User enters viewer address
`reading-handle`	Step 1 highlighted, spinner	Reading balance handles
`granting`	Step 2 highlighted, spinner	addViewer tx(s) pending
`confirmed`	Success badge + Arbiscan link	Access granted
`error`	Error message + retry	Failed at any step

Viewer Verification

After granting access, you can verify viewer status using `isViewer`:

```
function isViewer(bytes32 handle, address viewer) external view returns (bool);
```

```
const hasAccess = await publicClient.readContract({
  address: NOX_COMPUTE_ADDRESS,
  abi: noxComputeAbi,
  functionName: "isViewer",
  args: [handle, viewerAddress],
});
// Returns true if the viewer has been granted access
```

How the Viewer Decrypts

Once granted access, the viewer can decrypt the balance:

```
// Viewer's code (on their own machine)
import { HandleClient } from "@iexec-nox/handle";

const handleClient = new HandleClient({
  chainId: 421614,
  account: viewerWalletClient.account,
});

// 1. Read the handle
const handle = await publicClient.readContract({
  address: cRLCAddress,
  abi: confidentialTokenAbi,
  functionName: "confidentialBalanceOf",
  args: [ownerAddress],   // The token owner's address
});

// 2. Decrypt (only works if isViewer returns true)
const balance = await handleClient.decrypt(handle);
```

Critical Design Decisions

Per-Handle, Not Per-Token

Viewer access is granted on a **specific balance handle**, not on a token contract. This means:

- If the user wraps 100 cRLC and grants viewer access, the viewer can see the balance
- If the user then wraps 50 more cRLC, the balance handle **changes** → the viewer **loses access**
- The user must call `addViewer` again with the new handle

```
Wrap 100 cRLC → handle_A → addViewer(handle_A, auditor) ✅
Wrap 50 cRLC  → handle_B → auditor can't see handle_B ❌
                         → addViewer(handle_B, auditor) ✅
```

No Batch Support

Each handle requires a separate `addViewer` call. If the user has cRLC and cUSDC, that's two transactions (one per handle).

No Revoke (Yet)

There is currently no `removeViewer` function. Once access is granted on a handle, it persists until the handle changes (via wrap/unwrap/transfer). Revoke functionality is planned for a future release.

Multi-Token Support

The hook accepts an array of `TokenConfig[]`. For each token:

1. Read `confidentialBalanceOf` → get handle
2. Skip if handle is `ZERO_HANDLE`
3. Call `addViewer(handle, viewer)` for each non-zero handle

This allows granting a viewer access to **all** confidential balances in a single user action (though it's multiple on-chain transactions).

Gas Estimation

Estimated gas limit per `addViewer` call: ~100,000 gas units

Total gas depends on how many cTokens the user holds:

- 1 cToken: ~100k gas
- 2 cTokens: ~200k gas (two separate txs)

Important Notes

1. **Handle is ephemeral** — Every wrap, unwrap, or transfer changes the balance handle. Viewer access is tied to the handle, not the token. After any balance-changing operation, viewer access must be re-granted.

2. **NoxCompute is the ACL contract** — `addViewer` and `isViewer` are on the NoxCompute proxy (`0x5633...eA6`), not on the cToken contract itself.

3. **Read-only access** — Viewers can only decrypt (read) the balance. They cannot transfer, unwrap, or modify the balance in any way.

4. **2-second cooldown** — Required between sequential `addViewer` calls for NoxCompute processing.

5. **No self-viewer needed** — The token owner can always decrypt their own balance without calling `addViewer`.

6. **Not yet implemented:**

    - Listing current viewers for a handle
    - Revoking viewer access
    - Batch `addViewer` in a single transaction

Error Handling

Error	Context	Recovery
Invalid viewer address	Validation	Fix address in input
No confidential balances	Step 1	Wrap tokens first
All handles are zero	Step 1	Wrap tokens first
User rejected tx	Step 2	Reset and retry
addViewer reverted	Step 2	Check handle validity

Code Section (Dev Mode)

```
// Step 1: Read the current balance handle
bytes32 handle = cRLC.confidentialBalanceOf(ownerAddress);

// Step 2: Grant viewer access via NoxCompute
NoxCompute.addViewer(handle, viewerAddress);

// Verification (read-only, no tx needed)
bool hasAccess = NoxCompute.isViewer(handle, viewerAddress);
```